CITY CAR RACING

GAME DEVELOPMENT USING PYTHON AND TKINTER

AIM

To design and develop an interactive 2D car racing game using Python and Tkinter. The player controls a car using the left and right arrow keys, avoids incoming traffic, earns points by passing enemy cars, and restarts after a collision. The game includes a road environment, moving vehicles, score tracking, increasing difficulty, collision detection, game-over handling, and restart functionality.

PROCEDURE

1. Import Tkinter for the graphical interface and random for traffic placement.
2. Create the game window, score display, road canvas, and control buttons.
3. Set the screen size, road boundaries, lanes, car dimensions, and initial speed.
4. Draw the road, sidewalks, lane markings, player car, and enemy cars on the Canvas.
5. Use the left and right arrow keys to move the player's car between lanes.
6. Move enemy cars downward repeatedly to create the racing effect.
7. Return cars to random lanes after they leave the screen.
8. Increase the score when cars are passed and gradually increase traffic speed.
9. Detect collisions between the player car and enemy traffic.
10. Display Game Over and allow the player to restart using R or the Restart button.

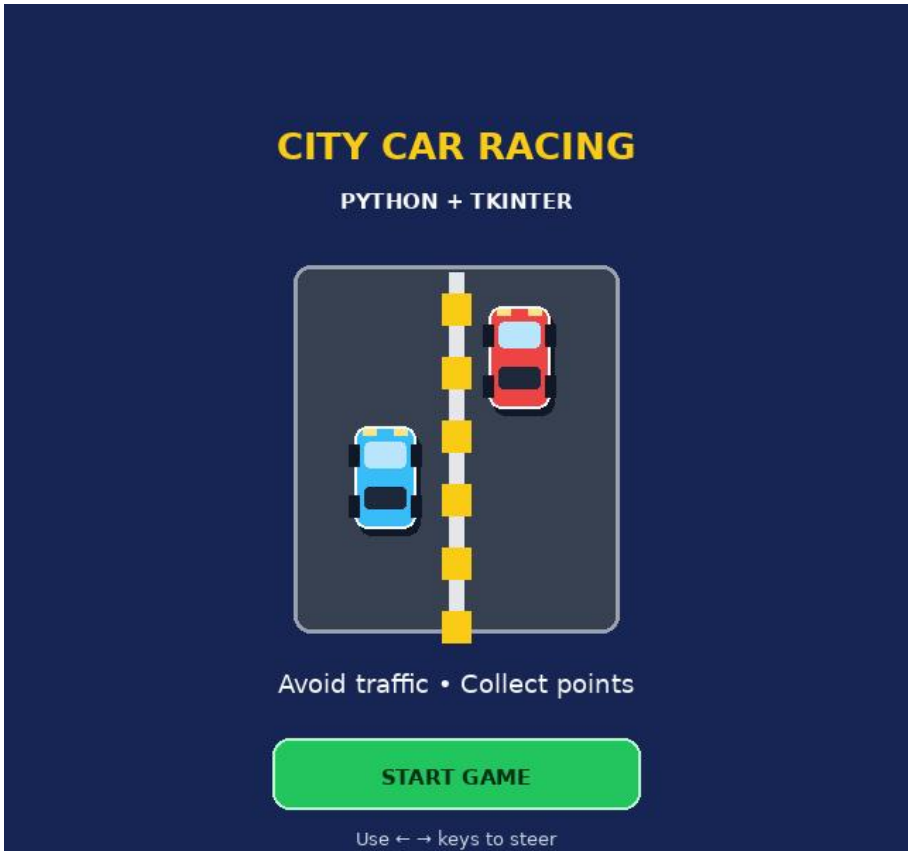
PROGRAM CODE

```
import tkinter as tk
import random
# -------------------- GAME SETTINGS --------------------
WIDTH = 600
HEIGHT = 620
ROAD_LEFT = 95
ROAD_RIGHT = 505
LANES = [165, 300, 435]
PLAYER_Y = 510
CAR_W = 42
CAR_H = 68
BG = "#172554"
ROAD = "#374151"
WHITE = "#F9FAFB"
YELLOW = "#FACC15"
BLUE = "#38BDF8"
RED = "#EF4444"
GREEN = "#22C55E"
class CityCarRacing:
    def __init__(self, root):
        self.root = root
        self.root.title("CITY CAR RACING - Python + Tkinter")
        self.root.resizable(False, False)
        self.root.configure(bg=BG)
        tk.Label(
            root, text="CITY CAR RACING",
            font=("Arial Black", 22), bg=BG, fg=YELLOW
        ).pack(pady=(10, 2))
        tk.Label(
            root, text="Avoid traffic • Collect points",
            font=("Arial", 11, "bold"), bg=BG, fg=WHITE
        ).pack(pady=(0, 8))
        self.hud = tk.Label(
            root, text="SCORE: 0    SPEED: 1",
            font=("Arial", 12, "bold"), bg=BG, fg=BLUE
        )
        self.hud.pack(pady=4)
        self.canvas = tk.Canvas(
            root, width=WIDTH, height=HEIGHT,
            bg=ROAD, highlightthickness=3,
            highlightbackground=BLUE
        )
        self.canvas.pack(padx=12, pady=8)
        controls = tk.Frame(root, bg=BG)
        controls.pack(pady=(0, 10))
        tk.Button(
            controls, text="START / RESTART",
            font=("Arial", 10, "bold"), bg=GREEN, fg="#052E16",
            command=self.restart, relief="flat", padx=12, pady=7
        ).pack(side="left", padx=5)
        tk.Button(
            controls, text="EXIT", font=("Arial", 10, "bold"),
            bg=RED, fg=WHITE, command=root.destroy,
            relief="flat", padx=16, pady=7
        ).pack(side="left", padx=5)
        self.root.bind("<Left>", lambda event: self.move_player(-1))
        self.root.bind("<Right>", lambda event: self.move_player(1))
        self.root.bind("<KeyPress-r>", lambda event: self.restart())
        self.root.bind("<KeyPress-R>", lambda event: self.restart())
        self.running = False
        self.after_id = None
        self.restart()
    def restart(self):
```

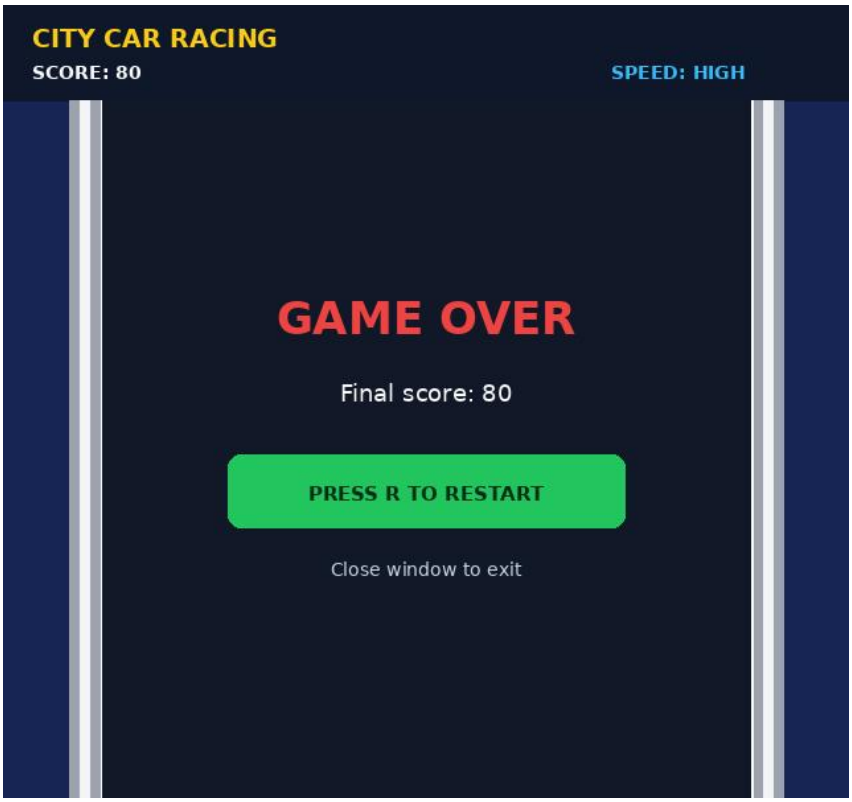
```
    if self.after_id is not None:
        self.root.after_cancel(self.after_id)
        self.after_id = None
    self.score = 0
    self.speed = 5
    self.player_lane = 1
    self.road_offset = 0
    self.running = True
    self.enemies = [
        {"lane": 0, "y": 135, "color": RED, "passed": False},
        {"lane": 2, "y": -100, "color": "#A855F7", "passed": False},
        {"lane": 1, "y": -330, "color": "#F97316", "passed": False},
    ]
    self.draw_scene()
    self.update_game()
def move_player(self, direction):
    if not self.running:
        return
    self.player_lane = max(0, min(2, self.player_lane + direction))
    self.draw_scene()
def draw_car(self, x, y, color, player=False):
    x1, y1 = x - CAR_W // 2, y - CAR_H // 2
    x2, y2 = x + CAR_W // 2, y + CAR_H // 2
    self.canvas.create_rectangle(
        x1 + 3, y1 + 5, x2 + 3, y2 + 5,
        fill="#111827", outline=""
    )
    self.canvas.create_rectangle(
        x1, y1, x2, y2, fill=color, outline=WHITE, width=2
    )
    self.canvas.create_rectangle(
        x1 + 7, y1 + 10, x2 - 7, y1 + 28,
        fill="#BAE6FD", outline="#0F172A", width=1
    )
    self.canvas.create_rectangle(
        x1 + 7, y1 + 43, x2 - 7, y1 + 56,
        fill="#1E293B", outline=""
    )
    for wx in (x1 - 4, x2 - 1):
        self.canvas.create_rectangle(wx, y1 + 13, wx + 5, y1 + 27,
                                     fill="#020617", outline="")
        self.canvas.create_rectangle(wx, y1 + 45, wx + 5, y1 + 59,
                                     fill="#020617", outline="")
    if player:
        self.canvas.create_rectangle(x1 + 5, y1 + 2, x1 + 14, y1 + 6,
                                     fill=YELLOW, outline="")
        self.canvas.create_rectangle(x2 - 14, y1 + 2, x2 - 5, y1 + 6,
                                     fill=YELLOW, outline="")
def draw_scene(self):
    self.canvas.delete("all")
    self.canvas.create_rectangle(0, 0, ROAD_LEFT, HEIGHT,
                                 fill="#9CA3AF", outline="")
    self.canvas.create_rectangle(ROAD_RIGHT, 0, WIDTH, HEIGHT,
                                 fill="#9CA3AF", outline="")
    self.canvas.create_rectangle(ROAD_LEFT, 0, ROAD_RIGHT, HEIGHT,
                                 fill=ROAD, outline="")
    self.canvas.create_line(ROAD_LEFT, 0, ROAD_LEFT, HEIGHT,
                            fill=WHITE, width=4)
    self.canvas.create_line(ROAD_RIGHT, 0, ROAD_RIGHT, HEIGHT,
                            fill=WHITE, width=4)
    for y in range(-70, HEIGHT, 90):
        yy = (y + self.road_offset) % (HEIGHT + 90) - 45
        self.canvas.create_rectangle(230, yy, 237, yy + 38,
                                     fill=YELLOW, outline="")
```

```
            self.canvas.create_rectangle(363, yy, 370, yy + 38,
                                         fill=YELLOW, outline="")
        for enemy in self.enemies:
            self.draw_car(LANES[enemy["lane"]], enemy["y"], enemy["color"])
        self.draw_car(LANES[self.player_lane], PLAYER_Y, BLUE, player=True)
        if not self.running:
            self.canvas.create_rectangle(
                ROAD_LEFT + 20, 210, ROAD_RIGHT - 20, 390,
                fill="#111827", outline=YELLOW, width=3
            )
            self.canvas.create_text(
                WIDTH // 2, 250, text="GAME OVER",
                font=("Arial Black", 25), fill=RED
            )
            self.canvas.create_text(
                WIDTH // 2, 295, text=f"Final score: {self.score}",
                font=("Arial", 16, "bold"), fill=WHITE
            )
            self.canvas.create_text(
                WIDTH // 2, 340, text="Press R or click RESTART",
                font=("Arial", 12, "bold"), fill=YELLOW
            )
        self.hud.config(text=f"SCORE: {self.score}    SPEED: {self.speed - 4}")
    def update_game(self):
        if not self.running:
            return
        self.road_offset = (self.road_offset + self.speed) % 90
        for enemy in self.enemies:
            enemy["y"] += self.speed
            if enemy["y"] - CAR_H // 2 > PLAYER_Y + CAR_H // 2:
                if not enemy["passed"]:
                    self.score += 10
                    enemy["passed"] = True
                    self.speed = min(12, 5 + self.score // 100)
            if enemy["y"] > HEIGHT + CAR_H:
                enemy["y"] = random.randint(-280, -90)
                enemy["lane"] = random.randrange(3)
                enemy["color"] = random.choice(
                    [RED, "#A855F7", "#F97316", "#EAB308"]
                )
                enemy["passed"] = False
            same_lane = enemy["lane"] == self.player_lane
            close_enough = abs(enemy["y"] - PLAYER_Y) < CAR_H - 8
            if same_lane and close_enough:
                self.running = False
                break
        self.draw_scene()
        if self.running:
            self.after_id = self.root.after(35, self.update_game)
        else:
            self.after_id = None
if __name__ == "__main__":
    root = tk.Tk()
    game = CityCarRacing(root)
    root.mainloop()
```

OUTPUT


CITY CAR RACING
PYTHON + TKINTER
Avoid traffic • Collect points
START GAME
Use ← → keys to steer


CITY CAR RACING
SCORE: 120
SPEED: HIGH



RESULT

Thus, the City Car Racing game was developed using Python and Tkinter. The player can steer the car with the keyboard, avoid incoming traffic, earn points for passing cars, and restart the game after a collision. The project demonstrates graphical user interface design, event handling, animation using the Tkinter after() method, random number generation, conditional statements, collision detection, and score management.